香港都會大學
HONG KONG METROPOLITAN UNIVERSITY

School of Science and Technology
MCOMPF - Master of Computing
COMP 8960SEF Capstone Project

An Intelligent Weather Forecast Notification System: WeatherWiseBot

Student Name: HE Xue
HKMU ID: 13927408
Supervisor: Dr. Jeff Au Yeung

Mar. 2026

Contents

Chapter 1. Introduction
- **1.1 Introduction**
- **1.2 Project Aim**
- **1.3 Project Objectives**
- **1.4 Value Propositions**

Chapter 2. Background Review
- **2.1 Review of Related Solutions for the Problem**
- **2.2 Highlight of the Proposed Solution**

Chapter 3. Preliminary Methodology
- **3.1 Overview**
- **3.2 Requirements, Supporting Technologies, and Technical Gap**
- **3.3 Architecture System Design**

An Intelligent Weather Forecast Notification System: WeatherWiseBot

Chapter 1. Introduction

1.1 Introduction

With the increasing frequency of extreme weather events due to climate change, personalized and timely weather information is essential for daily life, travel planning, and personal safety. Storms, heavy snow, and sudden rains happen more often, making weather more unpredictable.

Traditional weather apps are basic — users open them, check the forecast, and that's it. They do not send smart alerts based on user plans or location. This creates a significant gap: users need a system that actively pushes relevant information without requiring constant interaction.

This thesis presents WeatherWiseBot, an intelligent weather forecast notification system that delivers proactive, personalized updates directly to users' phones via SMS.

1.2 Project Aim

The aim of this project is to design WeatherWiseBot, an intelligent weather forecast notification system that makes weather information more personal and proactive. The system sends daily forecasts, outfit tips, and severe weather warnings by SMS, helping individuals make better daily plans while contributing to societal benefits such as fewer accidents and reduced costs.

1.3 Project Objectives

The objectives of this project are to:

1) Design a user-friendly web interface using Streamlit for seamless frontend (widget-driven forms) and backend (integrated scripting), allowing multi-region configurations, schedules, and event inputs.

2) Integrate a weather API for real-time, location-specific forecasts, warnings, and indices across regions.

3) Develop a scheduler for daily SMS forecasts at user-specified times, customized to primary or secondary locales.

4) Build an alert system that checks weather data every 15 minutes and sends instant SMS warnings for region-specific severe events.

5) Create a rule-based engine for clothing suggestions (e.g., if temperature < 10° C and rain chance > 30%, suggest "Wear a warm wool coat, scarf, and waterproof boots. Carry an umbrella").

6) Enable web event entry (e.g., "Flight: Shenzhen 2PM to Shanghai 4PM"), dispatching targeted SMS with origin/destination forecasts, tailored outfits, and risk warnings 24-48 hours in advance.

1.4 Value Propositions

For individuals, WeatherWiseBot saves time and improves safety. Commuters, travelers, and outdoor enthusiasts receive practical, schedule-aware alerts (e.g., outfit tips before a hike or flight delay warnings).

On a societal level, fewer weather-related accidents mean lower costs and better public preparedness. Stakeholders include end-users, meteorological data providers, and potentially healthcare systems (reduced cold-related illnesses through better clothing choices). Overall, WeatherWiseBot promotes a proactive approach to weather awareness in an era of climate uncertainty.

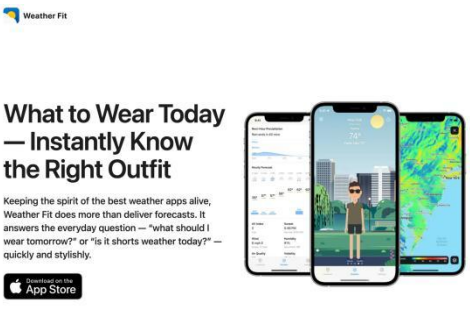
Chapter 2. Background Review

2.1 Review of Related Solutions for the Problem

Numerous weather applications and notification systems exist, providing valuable lessons for this project. Popular mobile apps like AccuWeather, MojiWeather, The Weather Channel, and WeatherBug dominate the market with features such as local forecasts, radar visualization, and push notifications for severe weather.

AccuWeather offers "MinuteCast" for minute-by-minute precipitation predictions, while MojiWeather provides accurate real-time weather updates, air quality indexes pushes and severe weather warnings for millions of locations. These systems have high user adoption due to intuitive interfaces than traditional weather forecast.

Specialized apps focus on niche personalization. Weather Fit provides weather-based clothing recommendations, suggesting layers, accessories, or rain gear based on temperature and precipitation. Apps like Carrot Weather add humorous notifications and basic personalization, while wardrobe tools integrate weather for outfit planning from user's digital closets.



However, most apps rely on apps push notifications, which can be disabled or missed if devices are silenced. Calendar integration is rare; few systems proactively link weather to user-scheduled events like flights or outings. Clothing recommendations

are often generic and not event-specific. SMS delivery is virtually absent, limiting accessibility for users preferring non-mobile channels.

2.2 Highlight of the Proposed Solution

WeatherWiseBot differentiates itself by using SMS as the primary notification channel and focusing on true personalization. Key highlights include:

1) Three main functions: Region-Specific Forecasts & Outfit Recommendations, Severe Weather Alert, and Schedule Plan Helper.
2) Rule-based "if-condition, then-result" outfit suggestions.
3) Instant 15-minute severe weather checks.
4) Web-based schedule input (e.g., "Flight: Shenzhen 2PM to Shanghai 4PM") with automatic dual-city reports via SMSs.

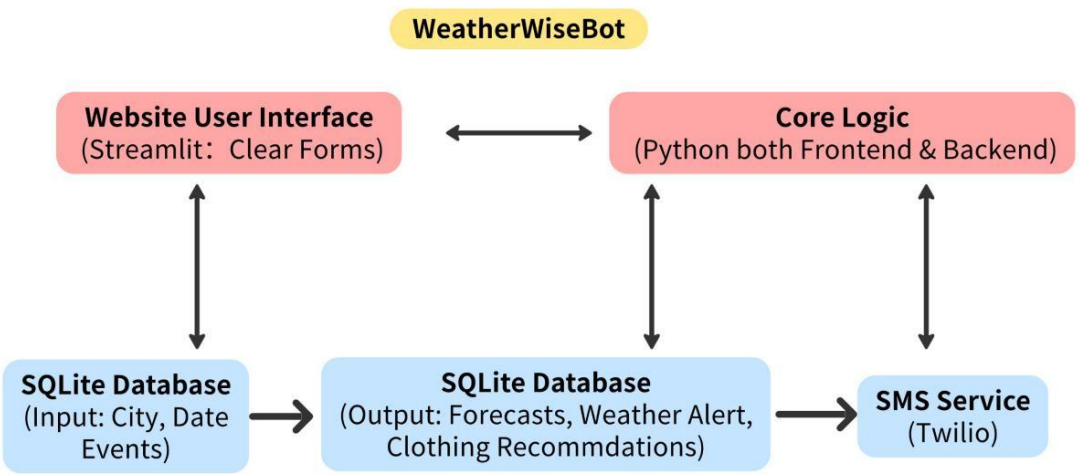
This positions WeatherWiseBot as a practical, user-centric solution that bridges the gaps in existing apps while adopting their strengths in data accuracy and alerting speed.

Chapter 3. Preliminary Methodology

3.1 Overview

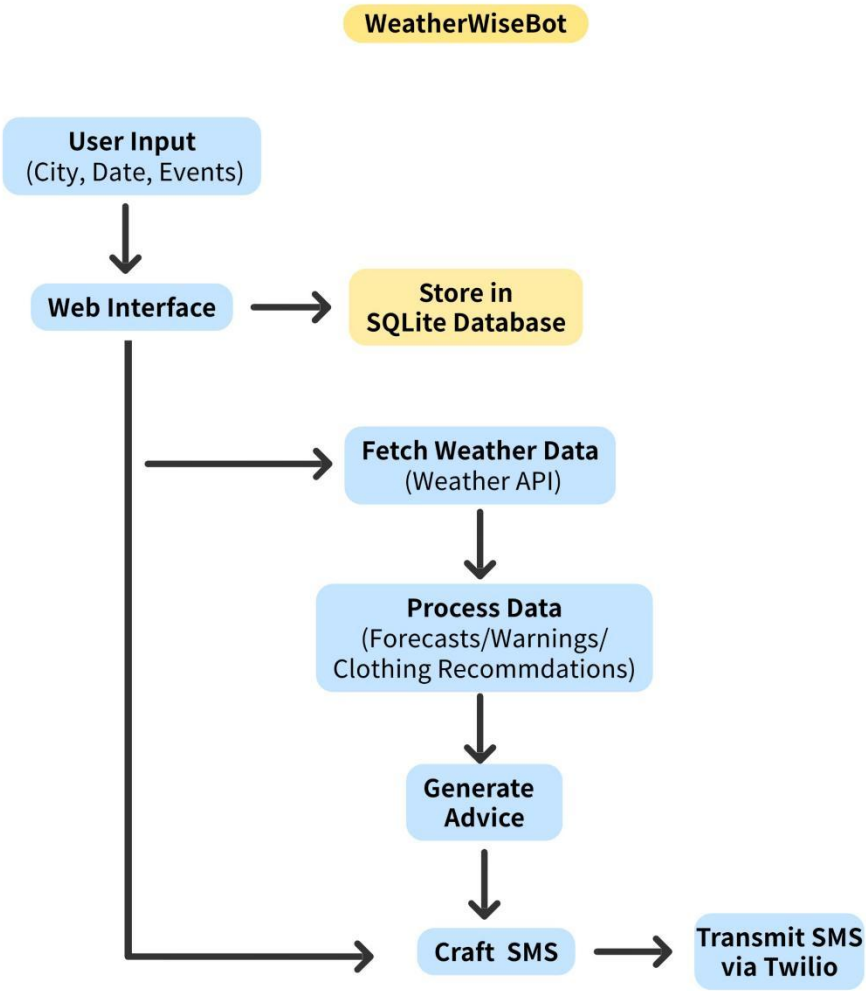
WeatherWiseBot operates as a web-based application with background processing for notifications. Users interact via a simple interface to configure settings, while the backend handles data processing and SMS delivery.

Component Diagram:



Website User Interface: Handles input and display.
Core Logic: Scheduling, recommendation engine.
Database: Persistent storage.
External Services: Weather API and Twilio SMS.

Data-Flow Diagram:



Data flows from user configuration to scheduled processing, triggering API calls and SMS outputs.

3.2 Requirements, Supporting Technologies, and Technical Gap

The project adopts an Agile Development methodology, combined with Waterfall Model for clear requirement definition. Top-down design starts from high-level features, refining stepwise.

3.2.1 Use-cases

1) User sets default cities and preferred notification time.
2) User adds calendar event (e.g., “Flight: Shenzhen 2PM to Shanghai 4PM”).
3) System sends daily SMS forecast.
4) System detects severe weather and sends instant alert.
5) Before a scheduled event, system sends targeted weather outlook, outfit suggestion, and risk warning.

3.2.2 Functional requirements

1) Support multi-city configuration and event input via web forms.
2) Retrieve real-time forecasts and warnings from weather API.

3) Generate and send scheduled daily SMS forecasts.
4) Detect severe conditions every 15 minutes and trigger instant alerts.
5) Provide rule-based clothing recommendations.

3.2.3 Non-functional requirements
1) Reliability: High uptime and graceful API failure handling.
2) Performance: API calls < 5 seconds, SMS delivery < 30 seconds.
3) Usability: Intuitive Streamlit interface.
4) Scalability: Start with single-user, extensible to multiple users.

3.2.4 Essential key technologies
1) Python: Core language for rapid development.
2) Streamlit: Web interface framework.
3) Weather API: Provides accurate forecasts.
4) SQLite: Lightweight database.
5) Twilio: SMS service.

Technical gaps: Limited AI for recommendations (rule-based initially); dependency on external API availability.

3.3 Architecture System Design

The system follows a layered architecture:
1) Presentation Layer: Streamlit – clean forms with sidebar for cities/events and session-state preview.
2) Application Layer: Same Python script handles both frontend and backend logic (scheduling, rule engine, event parser).
3) Data Layer: SQLite tables for regions, events, and forecasts.
4) Integration Layer: Requests for weather API; Twilio for SMS delivery.

Weather data fetching is isolated from SMS sending, allowing independent testing. Future extensions include machine learning for refined recommendations and multi-user support.

In conclusion, this preliminary design establishes a solid foundation for WeatherWiseBot, addressing identified gaps while building on proven technologies. Subsequent phases will focus on implementation and evaluation.